

KẾ HOẠCH TRIỂN KHAI CHI TIẾT

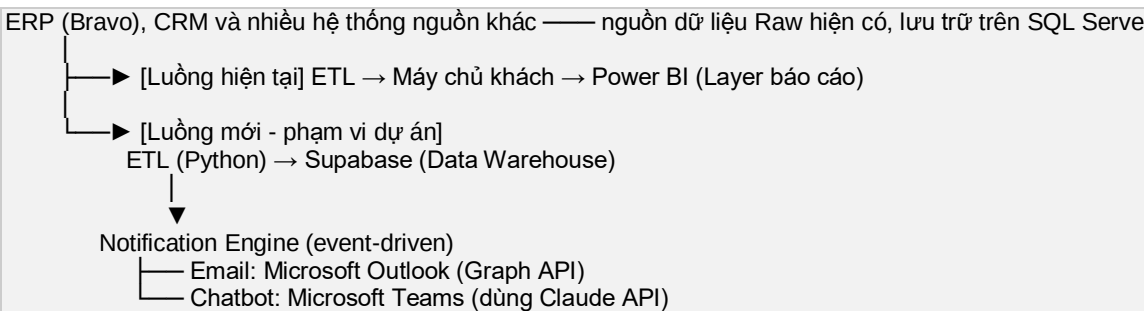
Hệ thống ETL Realtime + Data Warehouse + Notification / Chatbot tự động

Ngày lập: 02/07/2026

Phiên bản: 1.2

1. Tổng quan yêu cầu

Khách hàng cần xây dựng một luồng dữ liệu (workflow) hoạt động theo mô hình sau:



Giai đoạn TEST:

Email → thay bằng Gmail / Outlook test mailbox

Chatbot → thay bằng Telegram Bot

Mục tiêu chính: dữ liệu từ Bravo/SQL Server cần được đưa vào Supabase gần như real-time, sau đó tự động sinh thông báo (notification) và cho phép người dùng hỏi-đáp qua chatbot dựa trên dữ liệu này.

2. Hiện trạng dữ liệu (3 lớp hiện có)

Layer	Vai trò	Công nghệ	Tần suất cập nhật
Layer 1 – Raw	Dữ liệu gốc từ ERP/CRM	Bravo (SQL Server)	Theo giờ cố định trong ngày (nhiều lần/ngày)
Layer 2 – Staging	ETL đẩy dữ liệu sang máy chủ khách	ETL job hiện có của khách	Theo lịch, đồng bộ với Layer 1
Layer 3 – Báo cáo	Trực quan hoá	Power BI	Refresh theo lịch (Import mode), giới hạn số lần/ngày

Điểm mấu chốt: cả SQL Server (Layer 1) lẫn Power BI (Layer 3) hiện đều cập nhật theo lịch cố định (batch), không phải realtime tuyệt đối. Vì vậy khi khách hàng nói "kéo dữ liệu realtime song song với Power BI", cần hiểu đúng là near-real-time (NRT) — hệ thống mới cần bắt được thay đổi dữ liệu sớm nhất có thể, độc lập với lịch refresh của Power BI, mà không tạo thêm tải nặng lên SQL Server nguồn.

3. Phương án kéo dữ liệu Realtime / Near-Real-Time từ SQL Server

Có 4 phương án khả thi, xếp theo độ phức tạp tăng dần:

Phương án A — Polling theo watermark (khuyến nghị triển khai trước)

- ETL Python chạy định kỳ (5–15 phút/lần, tùy SLA khách yêu cầu) bằng scheduler (cron / Airflow / Windows Task Scheduler).
- Query dạng: `SELECT * FROM table WHERE UpdatedAt > @last_watermark`.
- Yêu cầu các bảng nguồn có cột thời gian cập nhật (UpdatedAt, ModifiedDate...). Nếu Bravo chưa có, cần đề xuất bổ sung hoặc dùng cột khoá tăng dần (ID, RowVersion).
- **Ưu điểm:** đơn giản, không cần quyền cao trên SQL Server, triển khai nhanh, rủi ro thấp.
- **Nhược điểm:** không phải realtime tức thời (có độ trễ = chu kỳ polling), không phát hiện được DELETE nếu không có cờ soft-delete.

Phương án B — SQL Server Change Tracking (CT)

- Tính năng có sẵn từ SQL Server 2008+, nhẹ hơn CDC, chỉ lưu "hàng nào đã đổi" (không lưu lịch sử giá trị).
- ETL truy vấn `CHANGETABLE(CHANGES ...)` để lấy danh sách thay đổi từ lần đồng bộ trước, sau đó JOIN lấy dữ liệu mới nhất.
- **Ưu điểm:** phát hiện chính xác cả INSERT/UPDATE/DELETE, tải nhẹ lên server nguồn.
- **Nhược điểm:** cần bật tính năng ở cấp database và table trên Bravo → cần xin quyền/đánh giá tác động với đội quản trị Bravo.

Phương án C — SQL Server Change Data Capture (CDC)

- Ghi lại đầy đủ lịch sử thay đổi (before/after values) vào các bảng hệ thống cdc.*.
- ETL đọc log LSN (Log Sequence Number) tăng dần để lấy chính xác từng thay đổi.
- **Ưu điểm:** đầy đủ nhất, phù hợp nếu sau này cần audit trail.
- **Nhược điểm:** cần SQL Server Standard/Enterprise có hỗ trợ CDC, tốn thêm tài nguyên I/O, cần quyền sysadmin để bật — khả năng cao phải phối hợp với đội IT/vendor Bravo.

Phương án D — Trigger + Queue table / Service Broker (event-driven, đẩy thay vì kéo)

- Tạo trigger trên bảng nguồn ghi ID bản ghi thay đổi vào bảng hàng đợi, hoặc dùng Service Broker để bắn message ngay khi có thay đổi.
- **Ưu điểm:** độ trễ thấp nhất, gần đúng nghĩa "realtime" nhất trong các phương án dùng SQL Server thuần.
- **Nhược điểm:** phải sửa schema/database của Bravo (tạo trigger) — rủi ro cao vì Bravo là phần mềm ERP thương mại đóng gói, có thể vi phạm điều khoản hỗ trợ hoặc bị ghi đè khi Bravo update version.

Khuyến nghị lộ trình

- **Giai đoạn 1 (MVP):** dùng Phương án A (watermark polling) với chu kỳ 5–15 phút — đáp ứng ngay nhu cầu "gần realtime" mà không động vào hệ thống Bravo.
- **Giai đoạn 2 (tối ưu, nếu cần độ trễ thấp hơn):** đánh giá khả năng bật Change Tracking (Phương án B) trên các bảng cụ thể cần theo dõi, sau khi được đội quản trị Bravo/SQL Server chấp thuận.
- **Không khuyến nghị** đụng vào trigger/CDC trực tiếp trên database lõi của Bravo trừ khi có xác nhận rõ ràng từ nhà cung cấp Bravo rằng việc này an toàn.

- **Chạy song song, không ảnh hưởng Power BI:** ETL mới nên dùng một kết nối/user SQL Server riêng, chỉ quyền đọc (read-only), và chạy độc lập theo lịch riêng — không phụ thuộc hay chia sẻ tài nguyên với job ETL đang đẩy dữ liệu cho Power BI.

4. Kiến trúc ETL & Data Warehouse (Python + Supabase)

4.1 Thành phần

- **Extract:** Python (pyodbc/pymssql) kết nối SQL Server Bravo, đọc theo watermark.
- **Transform:** chuẩn hoá kiểu dữ liệu, mapping trường, gộp bảng theo nhu cầu báo cáo/thông báo (dùng pandas hoặc SQL thuần trong Postgres sau khi load).
- **Load:** ghi vào Supabase (Postgres) qua Supabase Python client hoặc kết nối Postgres trực tiếp (psycopg2/SQLAlchemy), dùng upsert theo khoá chính để tránh trùng lặp.
- **Lưu trạng thái đồng bộ:** một bảng sync_watermark trong Supabase lưu thời điểm/ID đồng bộ gần nhất cho từng bảng nguồn — đảm bảo ETL chạy lại không bị lặp/mất dữ liệu (idempotent).
- **Scheduler:** cron job / Airflow / GitHub Actions scheduled workflow (tuỳ hạ tầng khách hàng cho phép) để chạy ETL định kỳ.

4.2 Tận dụng Supabase Realtime cho tăng thông báo

- Supabase (Postgres) hỗ trợ Realtime (dựa trên logical replication/WAL) và Database Webhooks.
- Khi ETL insert/update dữ liệu mới vào Supabase → cấu hình Database Webhook gọi thẳng đến Notification Engine (HTTP endpoint) ngay khi có thay đổi, thay vì để Notification Engine phải polling lại Supabase.
- Cách này giúp toàn bộ chuỗi "SQL Server → Supabase → Notification" trở thành event-driven, giảm độ trễ tổng thể xuống mức thấp nhất có thể trong giới hạn của phương án polling ở bước 1.

4.3 Thiết kế schema đề xuất (mô hình lớp dữ liệu trong Supabase)

- **raw_*:** dữ liệu thô, gần như nguyên bản từ Bravo (bronze layer).
- **stg_*:** dữ liệu đã chuẩn hoá, làm sạch (silver layer).
- **mart_*:** dữ liệu tổng hợp phục vụ trực tiếp cho notification/chatbot (gold layer).
- **sync_watermark, etl_run_log:** bảng vận hành, theo dõi tình trạng đồng bộ và log lỗi.

5. Hệ thống Notification & Chatbot

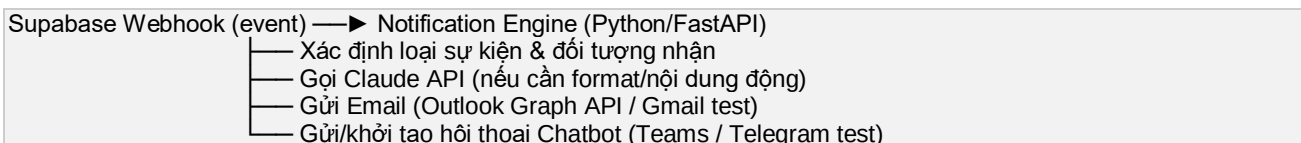
5.1 Notification qua Email (Outlook)

- **Production:** dùng Microsoft Graph API (sendMail), đăng ký ứng dụng trên Azure AD (App Registration) với quyền Mail.Send.
- **Test:** gửi thử qua Gmail SMTP/API hoặc một mailbox Outlook test riêng (sandbox) để không phát tán thông báo thật cho người dùng cuối trong giai đoạn kiểm thử.
- **Logic:** Notification Engine nhận sự kiện từ Supabase Webhook → format nội dung email (template) → gọi API gửi mail.

5.2 Chatbot trên Microsoft Teams (dùng Claude API)

- **Production:** đăng ký bot qua Azure Bot Framework, tạo Teams App Manifest, cấu hình endpoint bot trở về service backend.
- Backend nhận câu hỏi từ Teams → gọi Claude API (kèm ngữ cảnh dữ liệu truy vấn từ Supabase) → trả lời tự nhiên bằng ngôn ngữ người dùng.
- **Test:** dùng Telegram Bot API (tạo bot qua BotFather) để thử nghiệm luồng hội thoại, prompt, và tích hợp Claude API trước — vì Telegram setup nhanh, không cần phê duyệt phức tạp như Teams/Azure Bot. Sau khi luồng logic ổn định, port sang Teams SDK.

5.3 Kiến trúc Notification Engine



6. Lộ trình triển khai theo giai đoạn

Tháng/Tuần	Giai đoạn	Nội dung	Đầu ra
Tháng 1 Tuần 1-2	Khảo sát & chuẩn bị hạ tầng	Xin quyền đọc SQL Server (read-only), xác định bảng/trường cần lấy, tạo project Supabase, đăng ký Azure AD app	Danh sách bảng nguồn, tài khoản kết nối, môi trường Supabase
Tháng 1 Tuần 3-4	Xây ETL lõi	Viết pipeline Python (Extract–Transform–Load) theo watermark, thiết lập bảng sync_watermark, lịch chạy	ETL chạy ổn định, dữ liệu vào Supabase đúng, đồng bộ định kỳ
Tháng 2 Tuần 5-6	Notification Engine (bản test)	Xây webhook từ Supabase, gửi thử qua Gmail/Outlook test và Telegram	Luồng thông báo end-to-end hoạt động trên kênh test
Tháng 2 Tuần 7-8	Tích hợp Claude API cho Chatbot	Thiết kế prompt, kết nối dữ liệu từ Supabase làm ngữ cảnh trả lời, test hội thoại qua Telegram	Chatbot trả lời đúng dữ liệu, phản hồi hợp lý
Tháng 2 Tuần 7-8	Chuyển sang kênh Production	Cấu hình Outlook (Graph API) chính thức, đăng ký & publish Teams Bot	Hệ thống notification/chatbot chạy trên kênh thật
Tháng 3 Tuần 9-10	UAT & Go-live	Khách hàng kiểm thử thực tế, tinh chỉnh nội dung/logic, giám sát log lỗi	Nghiệm thu, chuyển giao vận hành
Tháng 3 Tuần 11-12	Giám sát & tối ưu sau go-live	Theo dõi độ trễ đồng bộ, tối ưu chu kỳ polling, đào tạo user, bàn giao tài liệu	Báo cáo vận hành, đề xuất cải tiến

7. Rủi ro & lưu ý cần thống nhất với khách hàng

- **Quyền truy cập SQL Server (Bravo):** cần tài khoản read-only riêng, tránh ảnh hưởng tới luồng ETL/Power BI hiện có.
- **Cột thời gian cập nhật:** cần xác nhận các bảng nguồn có cột UpdatedAt/tương đương; nếu không có, phải thống nhất phương án thay thế (ví dụ dùng ID tăng dần) hoặc đề xuất bổ sung.
- **Độ trễ thực tế:** cần thống nhất SLA cụ thể (ví dụ: "trong vòng 10 phút kể từ khi có dữ liệu mới") để chọn chu kỳ polling phù hợp, tránh kỳ vọng "realtime tức thời" không khả thi ở giai đoạn 1.

- **Bảo mật:** thông tin kết nối SQL Server, Supabase service key, Claude API key cần lưu trong secrets manager/biến môi trường, không hard-code.
- **Dữ liệu nhạy cảm:** nếu ERP/CRM có dữ liệu cá nhân (PII) hoặc tài chính, cần rà soát trường nào được phép đưa vào Supabase và nội dung nào được phép hiển thị qua notification/chatbot.
- **Chi phí vận hành:** Supabase (theo gói), Claude API (theo lượng token sử dụng), Azure Bot Framework/Teams (theo cấu hình đăng ký ứng dụng) — cần dự trù chi phí vận hành hàng tháng.
- **Phụ thuộc Bravo:** nếu sau này cần nâng cấp lên Change Tracking/CDC, phải làm việc với đội quản trị/nhà cung cấp Bravo để xin phép và đánh giá tác động.

8. Tóm tắt khuyến nghị kỹ thuật

- Dùng watermark polling (5–15 phút) cho giai đoạn đầu, không động vào schema Bravo.
- Dùng Supabase làm Data Warehouse trung tâm, tận dụng Database Webhooks/Realtime để biến bước "Supabase → Notification" thành event-driven, giảm độ trễ.
- Xây Notification Engine dùng chung một backend cho cả email và chatbot, chỉ khác kênh gửi (Outlook/Gmail, Teams/Telegram) — giúp dễ chuyển đổi giữa môi trường test và production mà không phải viết lại logic.
- Test toàn bộ luồng qua Gmail + Telegram trước, khi ổn định mới chuyển sang Outlook (Graph API) + Teams Bot Framework.
- Đánh giá nâng cấp lên Change Tracking ở giai đoạn 2 nếu SLA độ trễ yêu cầu khắt khe hơn mức polling có thể đáp ứng.